

Warning: interesting examples of RSA computations cannot realistically be performed by hand.

Ex:

• Key generation: Alice chooses primes $p=7$ and $q=13$.

She computes $n = p \cdot q = 91$ and

$$\phi(n) = (p-1)(q-1) = 72.$$

She then picks an e with $1 < e < 72$ and

$$\gcd(e, 72) = 1, \text{ say } e = 5.$$

She then computes $d \in \mathbb{Z}_{72}^{\times}$ as the inverse of $e \bmod 72$.

$$\text{As } 5 \cdot 29 = 145 = 2 \cdot 72 + 1,$$

$$\text{we have } d = 29.$$

• Bob retrieves Alice's public key $(n, e) = (91, 5)$.

In order to encrypt the message $m = 4 \in \mathbb{Z}_{91}$,

he computes

$$\begin{aligned} c &= m^e \bmod n = 4^5 \bmod 91 \\ &= 1024 \bmod 91 \\ &= 23 \end{aligned}$$

and sends $\boxed{c = 23}$ to Alice.

• Alice then decrypts

$$\begin{aligned} m &= c^d \bmod n = 23^{29} \bmod 91 \\ &= 4. \end{aligned}$$

↑
check it yourself

In practice, she does not compute the integer

$23^{29} (\approx 3 \times 10^{39})$. Note, for instance, the following:

a	1	2	3	4	5	6
$23^a \bmod 91$	23	74	64	16	4	1

computed easily using the recursive formula

$$x^{a+1} \bmod n = ((x^a \bmod n) x) \bmod n.$$

There are smarter ways of performing such calculations. $\rightarrow$ Modular exponentiation.

Security of RSA and integer factorisation

- Let (n, e) be Alice's public RSA Key.

Suppose that Oscar managed to factor

$$n = pq$$

into a product of primes p and q .

He can then duplicate the relevant parts of the Key generation step and derive Alice's private Key.

In detail:

Using the extended Euclidean algorithm, Oscar determines d as the modular inverse of e modulo $(p-1)(q-1)$.

In summary: deriving the private key (n, d) associated with a public RSA key (n, e) is at most as difficult as factoring n into a product of primes.

Note: We made implicit use of the Fundamental

Theorem of Arithmetic: every integer $m \geq 1$ can be written in the form

$$m = p_1 \cdots p_k$$

for $k \geq 0$ and primes $p_1, \dots, p_k$. Moreover, the factorisation

is unique up to the order of the primes.

- 89
- Conversely, suppose that Oscar knows both Alice's public RSA Key (n, e) and her corresponding private Key (n, d) .

There is an efficient (probabilistic) algorithm which allows him to (probably) factor n ; see e.g. Smart, lemma 11.6.

- Hence, the following computational problems are (essentially) equally difficult:

- Compute the private Key (n, d) from a public RSA Key (n, e) .
- Factor $n = pq$ for primes p and q .

(Remark: Knowledge of $\varphi(n) = (p-1)(q-1)$ also turns out to be equivalent to factoring n ; see Smart, lemma 11.7)

Q: how difficult is integer factorisation?

Integer factorisation problem:

Given an integer $n \geq 2$, find a proper divisor of n (i.e. a positive divisor distinct from 1 and n) or prove that n is prime.

Note: any algorithm for this can be recursively used to factor n into a product of primes.

Here's a naive(!) method we all know:

Trial Division (v1): For each $d = 2, \dots, n-1$, test if d divides n . If none of these numbers divides n , then n is prime.

Computational cost: $\sim n$ divisions

(We pretend that integer division takes unit time. This is not actually true!)

(Here's a minor, well-known improvement:)

Trial Division (v2): Same as before but only check
 $d = 2, \dots, \lfloor \sqrt{n} \rfloor$.

(Why does this suffice? Suppose that n is composite and write $n = pm$, where p is the smallest prime divisor of n . Then $p \leq m$ so that

$$p^2 \leq pm \leq n$$

and hence

$$p \leq \lfloor \sqrt{n} \rfloor$$

Computational cost: $\sim \sqrt{n}$ divisions

Is this "good" or "fast"?

the number of basic computational steps is bounded by a polynomial function in the size of the input

(Note: the 1) "time complexity".)

Ex: Given bit strings of lengths M and N representing non-negative integers, we can compute their sum using

$$\leq \max(M, N) + 1 \leq \underbrace{M + N + 1}_{\text{size of input}} \text{ additions.}$$

Ex: The "size" of an integer n in an algorithm is the number of binary (say) digits of n , i.e.

$$1 + \lfloor \log_2 n \rfloor \sim \log_2 n.$$

Trial division applied to a prime number n requires

$$\sim \sqrt{n} = n^{1/2} = 2^{\frac{1}{2} \log_2 n}$$

operations. This is exponential (not polynomial)

in the size ($\sim \log_2 n$) of the input and therefore not an efficient algorithm.

If you don't believe that trial division is bad, try using it to factor some of the Fermat numbers

$$F_k = 2^{2^k} + 1.$$

Integer factorisation: the state of the art

- It is unknown if a polynomial time algorithm for integer factorisation exists. If it does, then RSA is definitely insecure.

- The best known algorithms for integer factorisation have sub-exponential complexity

$$\sim L_n(\alpha, \beta) := e^{(\beta + o(1)) (\log n)^\alpha (\log \log n)^{1-\alpha}}$$

for $0 < \alpha < 1$. (For $\alpha = 0, 1$ we get polynomial and exponential complexity, resp.)

The General Number Field Sieve (GNFS) has

complexity $\sim L_n(1/3, 1.923)$.

Another famous method, the Elliptic-curve factorisation method (ECM) has complexity $\sim L_n(1/2, 1.42)$.

1987

These algorithms are based on sophisticated tools from number theory.

1988

- 95
- Shor's algorithm ¹⁹⁹⁴ is a polynomial time "quantum algorithm" for integer factorisation on a quantum computer.

The practical relevance remains unclear.

Still, it inspired the active field of

→ Post-quantum Cryptography

Even if integer factorisation were known to be provably hard, this would not necessarily imply that RSA is secure.

RSA Problem Given a public RSA Key (n, e) and $c \in \mathbb{Z}_n$, find $m \in \mathbb{Z}_n$ with $m^e \equiv c \pmod{n}$
(In other words: take e^{th} roots mod n .)

We saw: The RSA Problem is at most
as hard as integer factorisation.

But could it be strictly easier, i.e. could it
be possible to decrypt RSA messages without
ever knowing the private key?

We don't know!

Practical recommendation nowadays:

choose p and q of similar size (= bit length)
and $\log_2 n \geq 2048$.

Primality testing

We saw that integer factorisation may or may not be computationally hard — no efficient algorithm is known and the best methods available require deep mathematics.

The key generation step of RSA requires us to produce large, random primes ("prime generation").

Can we do that?

We first look at a related problem:

Can we efficiently test if a number is prime?

The Fermat primality test

Recall Fermat's little theorem:

If p is a prime and $a \in \mathbb{Z}$ with $\gcd(a, p) = 1$,

then $a^{p-1} \equiv 1 \pmod{p}$.

Hence, if $a^{n-1} \not\equiv 1 \pmod{n}$ for $n \geq 1$ and $a \in \mathbb{Z}$ with $\gcd(a, n) = 1$, then n is not prime.

This provides a way of proving that some numbers are composite without constructing a proper divisor: these numbers lack a property shared by all primes.

Fermat primality test:

Input: an integer $n \geq 2$

Output: message " n is definitely composite" or " n is probably prime"

- Pick a random integer a from $1, \dots, n-1$.
- If $a^{n-1} \not\equiv 1 \pmod{n}$, then n is definitely composite.
- Otherwise, if n passes the above test for "several" random values of a , then report that n is "probably prime".

Remarks: • If $n \geq 4$, then it is enough to choose a among $2, \dots, n-2$. (Exercise: explain!)

- While the Fermat primality test always correctly identifies primes, some composite numbers might be reported as "probably prime".